Table of Contents

1. Multi-Tenancy Models . . . 1
   1.1. One Tenant per Database (in a MongoDB Cluster) . . . 1
   1.2. Many Tenants in One Database (Shared Database) . . . 1
   1.3. Freemium and Trial Tenants . . . 1
   1.4. Tenant Onboarding and Access Invites . . . 2
      1.4.1. 1. Tenant provisioning: create the tenant itself . . . 2
      1.4.2. 2. Tenant onboarding workflow: define the journey after invite/registration . . . 3
      1.4.3. 3. Registration requests: capture onboarding data for approval . . . 5
      1.4.4. 4. Access invites: grant a user into an existing tenant . . . 5
      1.4.5. Invite extension points . . . 8
      1.4.6. How these pieces fit together . . . 8

Chapter 1. Multi-Tenancy Models

Quantum supports multiple multi-tenant models for MongoDB deployments:

1.1. One Tenant per Database (in a MongoDB Cluster)

- Each tenant is mapped to a dedicated MongoDB database within a cluster.
- Strong isolation at the database level; operational controls via MongoDB roles.
- Pros: Simplified backup/restore per tenant; reduced risk of data bleed.
- Cons: More databases to manage (indexes, connections), higher operational overhead.

How Quantum helps:

- DataDomain carries tenant identifiers (e.g., tenantId, ownerId, orgRefName) on each model.
- Repositories can resolve connections/DB selection per tenant, enabling routing to the appropriate database.

1.2. Many Tenants in One Database (Shared Database)

- Multiple tenants share a single database and collections.
- Isolation is enforced at the application layer using DataDomain filters.
- Pros: Fewer databases to manage; efficient index utilization and connection pooling.
- Cons: Strict discipline required to enforce filtering and access rules.

How Quantum helps:

- DataDomain is part of every persisted model, enabling programmatic, rule-based filtering.
- RuleContext and DomainContext can be used to inject tenant-aware filters into repositories and resources.
- Cross-tenant sharing can be modeled by specific DataDomain fields and RuleContext logic granting read access across tenants on a per-functional-area basis.

1.3. Freemium and Trial Tenants

- Programmatically create tenants to support self-service onboarding.
- Attach time-bound or capability-bound policies.
- Use scheduled jobs to convert/expire trials.

Quantum patterns:

- Tenant onboarding service creates a DataDomain scope and any default records.
- Policies are encoded in RuleContext checks to allow or restrict actions based on time, plan, or feature flags.

1.4. Tenant Onboarding and Access Invites

Quantum separates tenant creation from user access and from the user-facing onboarding journey. Those are related, but they are not the same thing:

- Tenant provisioning creates the tenant realm, runs migrations, and establishes the initial admin user.
- Tenant onboarding workflow defines the steps a tenant user should complete before access is considered fully activated.
- Access invites bring an individual user into an existing tenant and can attach scoped access metadata.

This split helps teams support both admin-driven setup and self-service onboarding without forcing every tenant into a single path.

1.4.1. 1. Tenant provisioning: create the tenant itself

The admin tenant provisioning API is exposed at:

- `POST /admin/tenants`
- `DELETE /admin/tenants/{realmId}`

Provisioning is intended for `admin` and `system` roles. The request creates the tenant catalog entry, initializes the tenant database, and ensures an initial admin credential exists.

Request fields:

Field	Purpose
`tenantDisplayName`	Human-readable label for the tenant
`tenantEmailDomain`	Source for the realm id; dots are converted to dashes
`orgRefName`	Organization reference used in the tenant `DomainContext` and `DataDomain`
`accountId`	Initial account number for the tenant
`adminUserId`	User id for the initial tenant admin
`adminSubject`	Optional stable subject override; defaults to `adminUserId`
`adminPassword`	Initial password for the admin user
`seedArchetypes` or `archetypes`	Optional list of seed archetypes to apply during tenant creation

Representative request:

```
{
  "tenantDisplayName": "Acme Logistics",
  "tenantEmailDomain": "acme.example",
  "orgRefName": "acme.example",
  "accountId": "1000000001",
```

```
  "adminUserId": "admin@acme.example",
  "adminSubject": "admin@acme.example",
  "adminPassword": "change-me-now",
  "seedArchetypes": ["DemoArchetype"]
}
```

Representative response:

```
{
  "realmId": "acme-example",
  "realmCreated": true,
  "userCreated": true,
  "appliedSeedArchetypes": ["DemoArchetype"],
  "warnings": []
}
```

What provisioning does:

- derives `realmId` from `tenantEmailDomain`
- writes a `Realm` catalog record in the system realm
- builds tenant `DomainContext` and `DataDomain`
- runs migrations in the new tenant realm
- creates or validates the initial admin user
- applies applicable base seed packs
- optionally applies requested seed archetypes
- applies indexes and verifies initialization

Provisioning is designed to be largely idempotent. If the same tenant or admin user already exists with matching attributes, the service proceeds and returns warnings instead of blindly recreating state. If existing state conflicts with the request, the service rejects the operation.

Deleting a tenant removes:

- the tenant database
- the tenant's `Realm` catalog entry
- matching credentials associated with that tenant setup

> `DELETE /admin/tenants/{realmId}` is destructive and is intentionally blocked for the configured system realm.

1.4.2. 2. Tenant onboarding workflow: define the journey after invite/registration

Tenant onboarding workflow configuration is exposed at:

- `GET /onboarding/workflow/current`
- `POST /onboarding/workflow/current`

This is tenant-scoped configuration. The active realm is derived from the current security context, so the workflow is saved and read per tenant realm.

The persisted model is `TenantOnboardingWorkflow`, which includes:

- `workflowEnabled`
- `inviteRequired`
- `registrationRequired`
- `surveyRequired`
- `adminApprovalRequired`
- `autoAssignSurveyOnInvite`
- `defaultSurveyRefName`
- `defaultInviteEmailTemplateKey`
- `workflowDefinitionJson`
- `completionMessage`

`defaultInviteEmailTemplateKey` is the email-template key the onboarding flow should use for invite messages. The framework default is `tenant-onboarding-invite`, which can be satisfied by:

- a classpath template under `templates/email/tenant-onboarding-invite/`
- a realm-scoped override in `email_templates`
- a workflow update that points to a different key

By default, the workflow steps are generated in this order:

- `invite`
- `registration`
- `survey`
- `approval`
- `activation`

Each step can be enabled or omitted by toggling the corresponding booleans. When no custom workflow JSON is supplied, the framework generates a default `workflowDefinitionJson` using those flags.

Representative response:

```
{
  "refName": "default-tenant-onboarding",
  "displayName": "Tenant User Onboarding",
```

```
  "activeStatus": true,
  "inviteRequired": true,
  "registrationRequired": true,
  "surveyRequired": true,
  "adminApprovalRequired": true,
  "autoAssignSurveyOnInvite": true,
  "defaultSurveyRefName": "tenant-user-onboarding",
  "defaultInviteEmailTemplateKey": "tenant-onboarding-invite",
  "completionMessage": "Your tenant onboarding is complete. You can now sign in and
begin using the workspace.",
  "steps": [
    { "key": "invite", "type": "access_invite", "required": true },
    { "key": "registration", "type": "registration_request", "required": true },
    { "key": "survey", "type": "survey", "required": true, "surveyRefName": "tenant-
user-onboarding" },
    { "key": "approval", "type": "admin_approval", "required": true },
    { "key": "activation", "type": "account_activation", "required": true }
  ]
}
```

This workflow config answers "what steps should happen?" It does not by itself send invites or create tenant users.

1.4.3. 3. Registration requests: capture onboarding data for approval

The registration-request endpoints are exposed at:

- `POST /onboarding/registrationRequest/create`
- `POST /onboarding/registrationRequest/approve`

This API is the registration step that the onboarding workflow can reference through the `registration_request` step type. It is useful when a tenant wants users or organizations to submit onboarding information before access is approved.

At a high level:

- `create` persists an `ApplicationRegistration`
- `approve` transitions an existing registration request through the repository approval logic

This is the structured "please review my onboarding submission" side of the journey, whereas access invites are the "you have been invited into this tenant" side.

1.4.4. 4. Access invites: grant a user into an existing tenant

Access invites are exposed at:

- `GET /access/invites`
- `GET /access/invites/{refName}`
- `POST /access/invites`

- `POST /access/invites/{refName}/revoke`
- `POST /access/invites/accept`

The first four require an authenticated caller in the target tenant. Invite acceptance is `@PermitAll` because the recipient may not yet have tenant access.

The persisted `AccessInvite` model includes:

- `email` or `targetUserId`
- `invitedByUserId`
- `scopeRefs`
- `grantedRoles`
- `allowedFunctionalAreas`
- `allowedFunctionalDomains`
- `allowedActions`
- `inviteMessage`
- `expiresAt`
- `acceptedAt`
- `acceptedUserId`
- `status` of `PENDING`, `ACCEPTED`, `REVOKED`, or `EXPIRED`

Representative invite creation request:

```
{
  "email": "jane.doe@partner.example",
  "scopeRefNames": ["north-america"],
  "grantedRoles": ["user"],
  "allowedFunctionalAreas": ["integration"],
  "allowedFunctionalDomains": ["exchange", "workflow"],
  "allowedActions": ["view", "create"],
  "expiresInDays": 14,
  "inviteMessage": "Join the tenant workspace for partner onboarding."
}
```

Representative creation response:

```
{
  "refName": "invite-1775100000000",
  "email": "jane.doe@partner.example",
  "scopeRefNames": ["north-america"],
  "grantedRoles": ["user"],
  "allowedFunctionalAreas": ["integration"],
  "allowedFunctionalDomains": ["exchange", "workflow"],
  "allowedActions": ["view", "create"],
```

```
  "status": "PENDING",
  "inviteToken": "raw-token-returned-at-create-time-only"
}
```

Important invite behavior:

- either `email` or `targetUserId` is required
- only one active pending invite is allowed for the same email or target user
- tokens are stored as a hash; the raw token is only returned at invite creation time
- invites expire automatically based on `expiresAt`
- invites can be revoked before acceptance

Representative accept request:

```
{
  "realm": "acme-example",
  "token": "raw-token-returned-at-create-time-only",
  "email": "jane.doe@partner.example",
  "firstName": "Jane",
  "lastName": "Doe",
  "password": "choose-a-password"
}
```

Representative accept response:

```
{
  "userId": "jane.doe@partner.example",
  "email": "jane.doe@partner.example",
  "defaultRealm": "acme-example",
  "inviteRefName": "invite-1775100000000",
  "grantedScopes": ["north-america"]
}
```

What happens when an invite is accepted:

- the invite token is validated and checked for expiry
- the user identity is resolved from the authenticated user, request payload, invite target user, or invite email
- if the user does not already exist, a new credential is created and a password is required
- if the user already exists, the service ensures the user is authorized for the tenant realm
- a `UserProfile` is created if needed
- the invite status is changed to `ACCEPTED`

1.4.5. Invite extension points

The invite flow is intentionally extensible through two SPIs:

- `AccessInviteProvisioner`
- `AccessInviteNotificationService`

`AccessInviteProvisioner` can:

- validate requested scopes when the invite is created
- perform application-specific provisioning when the invite is accepted

`AccessInviteNotificationService` can:

- deliver the invite token through email, messaging, or another notification mechanism

The framework ships with no-op default beans for both, so applications can opt in to deeper behavior without rewriting the invite flow.

1.4.6. How these pieces fit together

A common pattern looks like this:

1. An admin provisions the tenant with `POST /admin/tenants`.
2. The tenant configures or accepts the default onboarding workflow with `GET/POST /onboarding/workflow/current`.
3. A tenant admin creates an access invite for a user with `POST /access/invites`.
4. The invited user accepts the invite with `POST /access/invites/accept`.
5. If the workflow requires registration, survey, or approval, those steps are completed before the tenant considers onboarding complete.

This separation gives you flexibility:

- use tenant provisioning for admin-created tenants
- use registration requests for self-service intake and approval
- use access invites for controlled user admission into an existing tenant
- use workflow configuration to decide which of those steps are mandatory in each tenant